

Nastavení databáze:

- Otevřeme soubor `.env`
- `DATABASE_URL` podle vašeho databázového serveru
- Příklad použití: SQL je **MySQL** a vaše databáze se jmenuje **symfony_db**
`DATABASE_URL="mysql://username:password@localhost:3306/symfony_db"`
- Změňte `username`, `password` a `symfony_db` podle vašich údajů

Pozn pro xampp například

```
DATABASE_URL="mysql://pokus:pokus@127.0.0.1:3306/pokus?  
serverVersion=mariadb-10.4.28"
```

X. Tvorba entity:

- **symfony console make:entity Product** nebo **php bin/console make:entity Product**
- Následně můžete definovat Vaši entitu přímo v konzoli nebo dát **ENTER** a nadefinovat si jí celou sami

```

<?php

namespace App\Entity;

use App\Repository\ProductRepository;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: ProductRepository::class)]
class Product
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 50)]
    private ?string $name = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getName(): ?string
    {
        return $this->name;
    }

    public function setName(string $name): static
    {
        $this->name = $name;

        return $this;
    }
}

```

Sloupce id a name

- Tvorba migrace: **php bin/console make:migration**
- Provedení migrace: **php bin/console doctrine:migrations:migrate --no-interaction**

V phpmyadmin můžeme přidat data

pak rychle zjistíme, že tam data jsou

php bin/console dbal:run-sql 'SELECT * FROM product'

X. Výpis entity na stránku:

- Přidáme do HomeController cesty k entitě a Doctrineu
-

```
use Doctrine\Persistence\ManagerRegistry; //Přidáno pro manipulaci s entitami
use App\Entity\Product; //Přidáno pro práci s entitou "Product"
```

- Načteme si všechny záznamy do proměnný a tu si necháme vykreslit na vybranou stránku
-

```
#[Route('/home', name: 'app_home')]
public function index(ManagerRegistry $doctrine): Response
{
    // najít všechny produkty
    $products = $doctrine->getManager()->getRepository(Product::class)->findAll();

    return $this->render('home/index.html.twig', [
        'controller_name' => 'HomeController',
        'products' => $products
    ]);
}
```

-
- Upřesnění controlleru

```
<?php

namespace App\Controller;

use Doctrine\Persistence\ManagerRegistry;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use App\Entity\Product;
class HomeController extends AbstractController
{
    #[Route('/home', name: 'home')]
    public function Home(ManagerRegistry $doctrine): Response
    {
        $products=$doctrine->getManager()-
        >getRepository(Product::class)->findAll();
        return $this->render('home.html.twig', ['products'=>$products]);
    }
}
```

-
-
- Na stránce si pak jednotlivé záznamy vypíšeme přes for cyklus

```
{% for product in products %}
  <tr>
    <td> {{product.getId()}} </td>
    <td> {{product.getName()}} </td>
  </tr>
{% endfor %}
```

-
- Twig pak je

- ```
{% for product in products %}
{{ product.getId() }}
{{ product.getName() }}

{% endfor %}
```

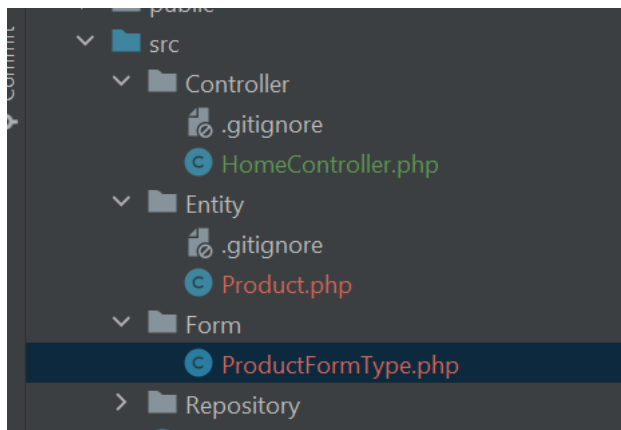
## X. Tvorba formuláře:

### Vytvoření formuláře:

**php bin/console make:form ProductForm**

- 

Vytvoří se soubor ProductFormType.php



přidejte entitu a volitelně další datové typy (pokud nejsou přidáné)

soubor ProductFormType.php

- 

```
use App\Entity\Product; //Název entity
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
use Symfony\Component\Form\Extension\Core\Type\TextType; // Přidá další datové typy
```

- 

```
class ProductFormType extends AbstractType
{
 public function buildForm(FormBuilderInterface $builder, array $options): void
 {
 $builder
 ->add('name', TextType::class, ['label' => 'Název']); //Specifikuje název pod kterým se to bude zobrazovat
 }

 public function configureOptions(OptionsResolver $resolver): void
 {
 $resolver->setDefaults([
 'data_class' => Product::class,
]);
 }
}
```

- **Prakticky**

- `?php`

```
namespace App\Form;
```

```

use App\Entity\Product;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;

class ProductFormType extends AbstractType
{
 public function buildForm(FormBuilderInterface $builder,
array $options): void
 {
 $builder
 ->add('name')
 ;
 }

 public function configureOptions(OptionsResolver
$resolver): void
 {
 $resolver->setDefaults([
 'data_class' => Product::class,
]);
 }
}

```

- Následně přidáme další metodu do **HomeController**

```

#[Route('/product/new', name: 'product_new')] //cesta
public function new(Request $request, ManagerRegistry $doctrine): Response
{
 $product = new Product(); //Tvorba nového objektu typu Product

 $form = $this->createForm(ProductFormType::class, $product); //Tvorba nového formuláře podle vzoru ProductFormType

 $form->handleRequest($request); //Předání dat z formuláře

 if ($form->isSubmitted() && $form->isValid()) // Kontrola validity a odeslání
 {
 $product = $form->getData();

 $em = $doctrine->getManager();

 $em->persist($product);
 $em->flush(); //Přidání data do databáze

 return $this->redirect('/'); //Přesměrování na Homepage
 }

 return $this->render('home/product_new.html.twig', [//Stránka s daným formulářem
 'form' => $form->createView(),
]);
}

```

Prakticky HomeController.php

```

namespace App\Controller;

use App\Form\ProductFormType;
use Doctrine\Persistence\ManagerRegistry;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

```

```
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use App\Entity\Product;
class HomeController extends AbstractController
{
```

.....

```
#[Route('/product/new', name: 'productnew')]
public function new(Request $request, ManagerRegistry $doctrine): Response
{
 $product=new Product();
 $form=$this->createForm(ProductFormType::class,$product);
 $form->handleRequest($request);

 if ($form->isSubmitted() && $form->isValid())
 {

 $products=$form->getData();
 $em=$doctrine->getManager();
 $em->persist($products);
 $em->flush();
 return $this->redirect('/home');
 }

 return $this->render('product_new.html.twig',['form'=>$form-
>createView()]);
}
```

- Vytvoříme stránku **product\_new.html.twig**
- 

```
{% extends 'base.html.twig' %}

{% block title %}Hello HomeController!{% endblock %}

{% block body %}
<style>
 .example-wrapper { margin: 1em auto; max-width: 800px; width: 95%; font: 18px/1.5 sans-serif; }
 .example-wrapper code { background: #F5F5F5; padding: 2px 6px; }
</style>
{{ form_start(form) }} <!-- Název formuláře jako předáváme v HomeController-->
{{ form_row(form.name) }}
{{ form_row(form.productType) }} <!--Názvy polí podle toho jak jsou definovaná ve ProductFormType-->
<button type="submit">Save</button>
{{ form_end(form) }}

{% endblock %}
```

## Prakticky

```
{% block body %}

 {{ form_start(form) }}
 {{ form_row(form.name) }}
 <button type="submit">Save</button>
 {{ form_end(form) }}

{% endblock %}
```

## X. Zbylé CRUD operace:

- Přidáme metody podle vybrané operace
- **Edit**
- 

```
#[Route('/product/edit/{id}', name: 'product_edit')]
public function edit(Request $request, ManagerRegistry $doctrine, $id): Response //Přidá parametr id pro zjištění záznamu
{
 $product = $doctrine->getManager()->getRepository(Product::class)->find($id); //Najde na základě id záznam
 if (!$product) {
 throw $this->createNotFoundException('No product found for id ' . $id); //Error pokud není záznam nalezen
 }

 $form = $this->createForm(ProductFormType::class, $product); //Tvorba nového formuláře podle vzoru ProductFormType

 $form->handleRequest($request); //Předání dat z formuláře

 if ($form->isSubmitted() && $form->isValid()) {
 $em = $doctrine->getManager(); // Objekt pro práci s entitami

 $em->flush(); // Provedení změn v databázi

 return $this->redirectToRoute('app_home');
 }

 return $this->render('home/product_edit.html.twig', [
 'form' => $form->createView(),
]);
}
```

```
Prakticky #[Route('/product/edit/{id}', name: 'product_edit')]
public function edit(Request $request, ManagerRegistry $doctrine, $id):
Response
{
 $product=$doctrine->getManager()->getRepository(Product::class)-
>find($id);
 if (!$product)
 {
 throw $this->createNotFoundException('Produkt s tímto číslem
nenalezen ' . $id);
 }
 $form=$this->createForm(ProductFormType::class,$product);
 $form->handleRequest($request);
 if ($form->isSubmitted() && $form->isValid())
 {

 $em=$doctrine->getManager();
 $em->flush();
 return $this->redirect('/home');
 }

 return $this->render('product_edit.html.twig', ['form'=>$form-
>createView()]);
}
```

- Vytvoříme html stránku na které se bude vytvářet formulář

```

<div class="container"> {# Wrap the form in a container to center it #}
 <div class="row justify-content-center"> {# Use Bootstrap to horizontally center the content #}
 <div class="col-md-8"> {# Adjust the column width as needed #}
 <h1>Záznam nového produktu</h1>

 {{ form_start(form, {'attr': {'class': 'form-horizontal'}}) }}
 <div class="form-group">
 {{ form_label(form.name, 'Název', {'label_attr': {'class': 'col-form-label col-sm-2'}}) }}
 <div class="col-sm-10">
 {{ form_widget(form.name, {'attr': {'class': 'form-control'}}) }}
 {{ form_errors(form.name) }}
 </div>
 </div>

 <div class="form-group">
 {{ form_label(form.productType, 'Typ', {'label_attr': {'class': 'col-form-label col-sm-2'}}) }}
 <div class="col-sm-10">
 {{ form_widget(form.productType, {'attr': {'class': 'form-control'}}) }}
 {{ form_errors(form.productType) }}
 </div>
 </div>

 <div class="form-group">
 <div class="col-sm-offset-2 col-sm-10">
 <button type="submit" class="btn btn-primary">Uložit</button>
 Zpět
 </div>
 </div>
 {{ form_end(form) }}
 </div>
 </div>
</div>

```

Zjednodušeno

```

{% block body %}
 <h1>Oprava produktu</h1>

 {{ form_start(form) }}
 {{ form_widget(form.name) }}
 {{ form_errors(form.name) }}

 <button type="submit">Save</button>
 Home
 {{ form_end(form) }}
{% endblock %}

```

- Delete

- 

```
#[Route('/product/delete/{id}', name: 'product_delete')]
public function delete(Request $request, ManagerRegistry $doctrine, $id): Response
{
 $product = $doctrine->getManager()->getRepository(Product::class)->find($id); //Najde na základě id záznam

 if (!$product)
 {
 throw $this->createNotFoundException('No product found for id ' . $id); //Error pokud není záznam nalezen
 }

 if ($this->isCsrfTokenValid('delete'.$product->getId(), $request->request->get('_token')))
 {
 $em = $doctrine->getManager();
 $em->remove($product); //Odebere záznam
 $em->flush(); // Provedení změn v databázi

 return $this->render('home/index.html.twig', [//Cesta jakou stránku má kontroler vykreslit
 'controller_name' => 'HomeController',
 'variable_test' => 'Value_test', //Název proměné a její hodnota
]);
 }

 return $this->render('home/product_delete.html.twig', [
 'product' => $product,
]);
}
```

## Prakticky

```
#[Route('/product/delete/{id}', name: 'product_delete')]
public function delete(Request $request, ManagerRegistry $doctrine, $id): Response
{
 $product=$doctrine->getManager()->getRepository(Product::class)->find($id);
 if (!$product)
 {
 throw $this->createNotFoundException('Produkt s tímto číslem nenalezen '.$id);
 }

 if ($this->isCsrfTokenValid('delete'.$product->getId(), $request->request->get('_token')))
 {
 $em=$doctrine->getManager();
 $em->remove($product);
 $em->flush();
 return $this->render('index.html.twig', [
 'controllername'=>"HomeController",
 'variable_test'=>'Value_Test'
]);
 }

 return $this->render('delete.html.twig', ['product'=>$product]);
}
```

Poznámka (csrf): <https://www.root.cz/clanky/uvod-do-frameworku-symfony-jednoduche-formulare/>

- Vytvoříme html stránku na které se bude vykreslovat záznam, který budeme chtít smazat
- 

```
<div class="row justify-content-center">
 <div class="col-auto">
 <table class="table table-bordered">
 <thead>
 <tr>
 <th scope="col">Id</th>
 <th scope="col">Name</th>
 <th scope="col">Type</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th scope="row"> {{ product.getId() }} </th>
 <td> {{ product.getName() }} </td>
 <td> {{ product.getProductType().getName() }}</td>
 </tr>
 </tbody>
 </table>

 <form action="{{ path('product_delete', {'id': product.getId() }) }}" method="post">
 <input type="hidden" name="_token" value="{{ csrf_token('delete' ~ product.getId()) }}">
 <button type="submit" class="btn btn-primary">Delete</button>
 </form>

 <button class="btn btn-primary" onclick="window.history.back();">Go Back</button>
 </div>
</div>
```

## Prakticky

```
{% block body %}

 {{ product.getId()}}

 {{ product.getName()}}

 <form action="{{ path('product_delete', {id:product.getId()}) }}"
 method="post">
 <input type="hidden" name="_token" value="{{ csrf_token('delete'
~product.getId()) }}" >

 <button type="submit" > Delete </button>

 </form>

 <button onclick="window.history.back();" > zpet </button>

{% endblock %}
```

## index.html.twig

```
Smazano
{{ controllername }}
{{ variable_test }}

```

